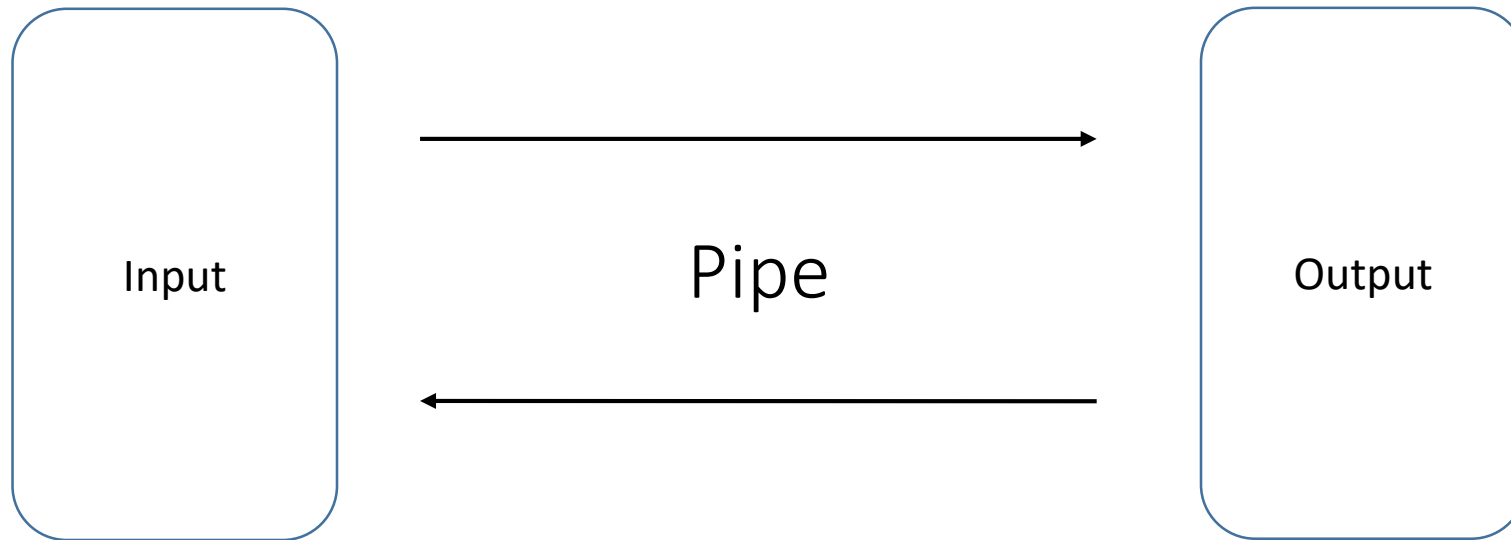


A thread-safe pipe implementation

Evangelos Zampras | Evangelos Balamotis

ECE, University of Thessaly



Αποφυγή των Race Conditions

Εντοπίσαμε race conditions (ταυτόχρονη εγγραφή και ανάγνωση από/προς σωλήνα), που ουσιαστικά αφορούν την προστασία του count variable μέσα στο circular buffer.

Για να τα αποφύγουμε εισάγουμε μεταβλητές read, write και turn για να δηλώσουμε πότε μια υπορουτίνα θέλει να διαβάσει, να γράψει στον κάθε ένα σωλήνα και πότε είναι η σειρά του read και του write.

Η υλοποίηση είναι παρόμοια με τους αλγορίθμους Dekker και Peterson που αποτελούν αποδεδειγμένες λύσεις στο πρόβλημα του mutual exclusion.

Συναρτήσεις βιβλιοθήκης

Κατασκευάσαμε τις εξής συναρτήσεις ως “API” της βιβλιοθήκης:

Συνάρτηση	Τιμή επιστροφής	Χρήση
pipe_read()	1: success, 0: pipe closed, -1: pipe not found	Διάβασμα 1 byte από αγωγό
pipe_write()	1: success, -1: pipe not found	Γράψιμο 1 byte σε αγωγό
pipe_open()	<id>: success, -1: pipe not created	Άνοιγμα νέου αγωγού
pipe_writeDone()	1: success, -1: pipe not found	Κλείσιμο αγωγού για γράψιμο
pipe_init()	void	Δημιουργία εσωτερικής δομής pipe system
pipe_destroy()	void	Εκκαθάριση αγωγού και αποδέσμευση μνήμης

Υλοποίηση βιβλιοθήκης

Η βιβλιοθήκη μας λειτουργεί εσωτερικά με βάση πίνακες που αποθηκεύουν δεδομένα για τον κάθε αγωγό. Τα δεδομένα αυτά είναι:

- Αν ο αγωγός υπάρχει (Το id του αν υπάρχει, -1 αν δεν υπάρχει)
- Αν ο αγωγός είναι ανοιχτός (1 ή 0)
- Αν κάποιο thread επιθυμεί γράψιμο
- Αν κάποιο thread επιθυμεί διάβασμα
- Η τιμή της μεταβλητής turn (σειρά του thread για διάβασμα ή γράψιμο, με βάση τον αλγόριθμο Peterson)
- Οι διευθύνσεις των εσωτερικών buffers ενός αγωγού, αν αυτός έχει δημιουργηθεί
- Το συνολικό πλήθος αγωγών που έχουμε δημιουργήσει

Η υλοποίηση λειτουργεί σωστά με αυτά τα δεδομένα καθώς έχουμε τον περιορισμό του specification ότι μόνο ένα thread γράφει και μόνο ένα thread διαβάζει έναν αγωγό κάθε στιγμή.

Testbenches

Έχουμε δημιουργήσει δύο testbenches με διαφορετικούς στόχους, για τη δοκιμή της λειτουργικότητας της βιβλιοθήκης αγωγών. (\$ make για να δημιουργηθούν και τα δύο)

./main (Source: main.c): Testbench που περιγράφεται στο cp_assignments_1.pdf

Δημιουργία δύο αγωγών 64 byte, «αποστολή» ενός αρχείου (input1) από ένα thread μέσω ενός αγωγού (p1) και «παραλαβή» του αρχείου από το άλλο thread. Το thread 2 αποθηκεύει το αρχείο σε αντίγραφο (output1).

Στη συνέχεια, μεταφορά του ίδιου αρχείου προς την ακριβώς αντίθετη κατεύθυνση μέσω ενός δεύτερου αγωγού (p2) και αποθήκευση ενός δεύτερου αντιγράφου (output2).

./main2 (Source: main2.c): Testbench με 5 αγωγούς που λειτουργούν ταυτόχρονα.

Δημιουργία 10 threads, τα 5 γράφουν σε 5 αγωγούς και τα υπόλοιπα διαβάζουν από τους αντίστοιχους αγωγούς. Ως αποτέλεσμα, 5 αρχεία input «αντιγράφονται» σε 5 αρχεία output.

Σημ.: Τα αρχεία εισόδου και εξόδου σε κάθε περίπτωση πρέπει να υπάρχουν ήδη πριν την έναρξη του testbench.

Testbenches

- Τα testbenches εκτυπώνουν παραπάνω πληροφορίες για τη λειτουργία των threads (πότε ένα thread ξεκινά/σταματά, κ.λπ.)
- Η λειτουργία δοκιμάστηκε με αρχεία διαφόρων μεγεθών (από μερικά KB έως και 5.3 MB), ώστε να διαπιστωθεί ότι οι πληροφορίες γράφονται σωστά μέσω των pipes.
- Πρέπει να κληθεί η `pipe_init()` πριν τη δημιουργία αγωγών!

A thread-safe pipe implementation

Evangelos Zampras | Evangelos Balamotis

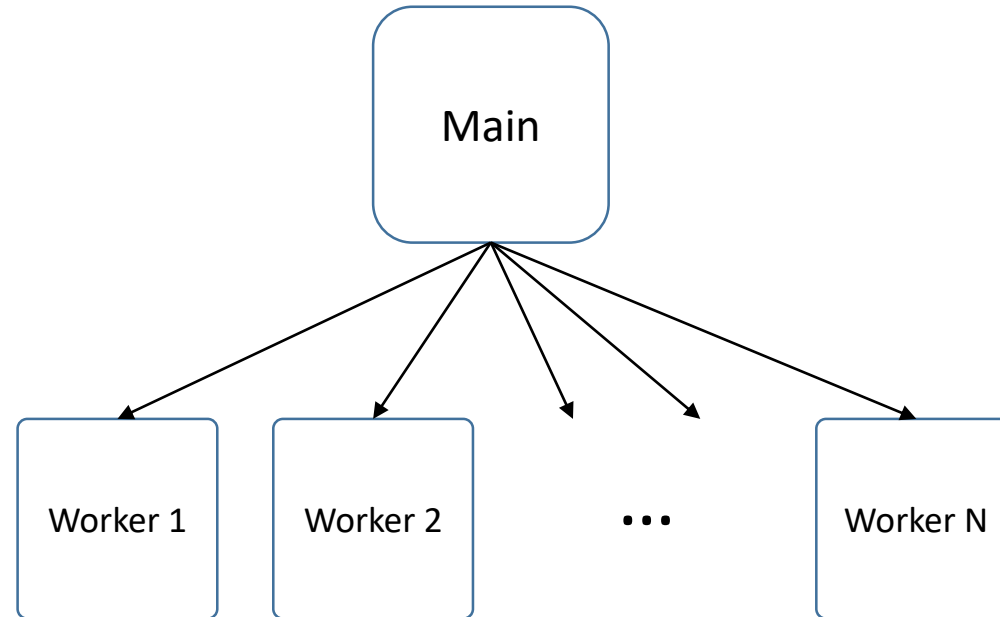
ECE, University of Thessaly

Thanks for your time!

Distributed primes calculation

Evangelos Zampras | Evangelos Balamotis

ECE, University of Thessaly



Υλοποίηση multithreading

Το πρόγραμμα λαμβάνει ως όρισμα τον αριθμό των threads, δημιουργεί τα threads και στη συνέχεια λαμβάνει ως είσοδο ακέραιους, για καθέναν από τους οποίους υπολογίζει αν είναι prime και εκτυπώνει <number>:<is_prime> όπου <number> η τιμή εισόδου και <is_prime> μια τιμή (1 ή 0) που υποδεικνύει αν η τιμή εισόδου είναι prime ή όχι.

Το πρόγραμμα σταματά την ανάγνωση όταν ανιχνεύεται **EOF**.

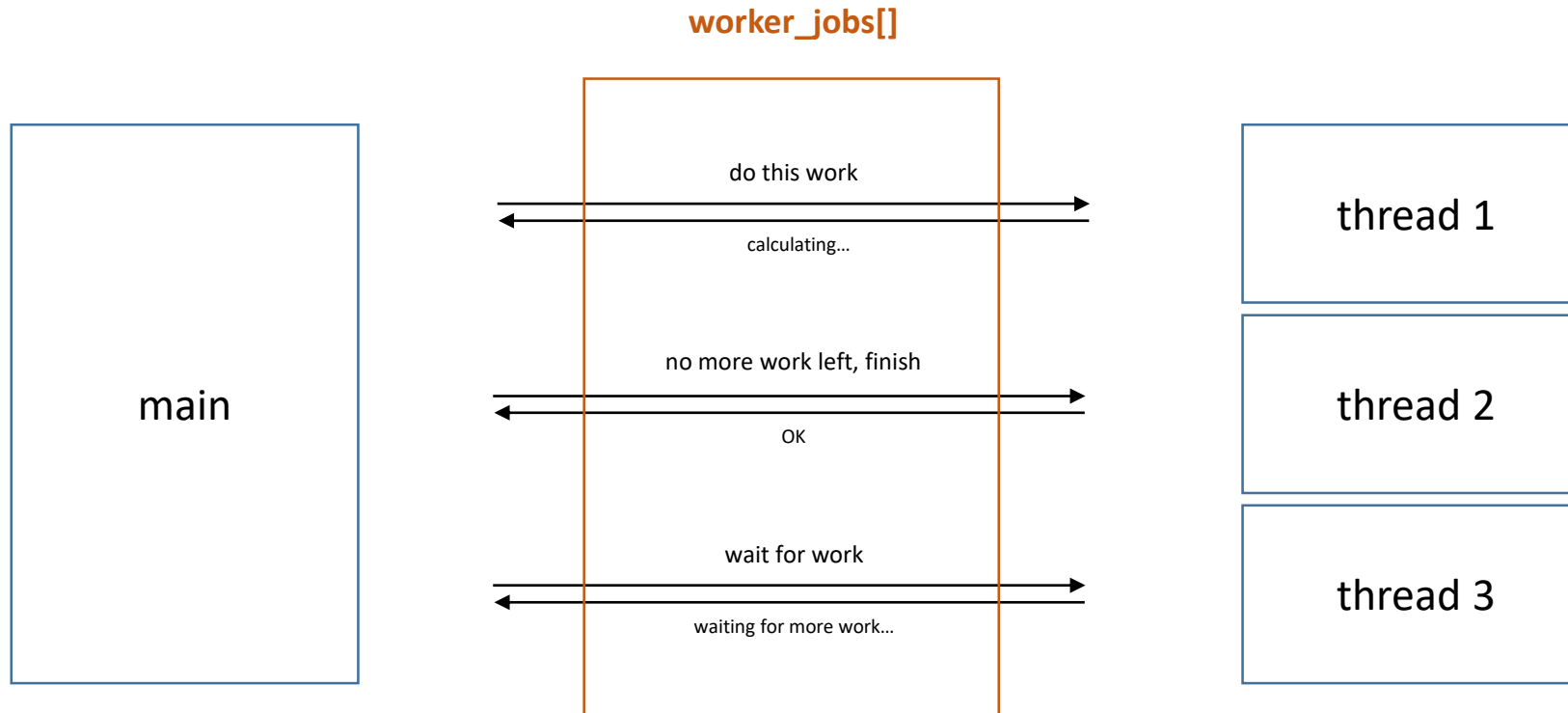
Υλοποίηση multithreading

Η υλοποίηση του multithreading για επιτάχυνση των υπολογισμών γίνεται ως εξής:

1. Η main διαβάζει μια τιμή εισόδου
2. Ελέγχει τα worker threads, και ανιχνεύει ποιο από αυτά περιμένει να λάβει καινούριο workload. (i.e. έχει κατάσταση waiting)
3. Δίνει την τιμή στο worker, και θέτει την κατάστασή του σε not-waiting ώστε εκείνο να αρχίσει να επεξεργάζεται την τιμή
4. Όταν το thread ολοκληρώσει τους υπολογισμούς, εκτυπώνει το αποτέλεσμα (μόνο ένα thread επεξεργάζεται μία τιμή κάθε φορά, οπότε δεν υπάρχει ambiguity ή race conditions ως προς το αποτέλεσμα που υπολογίζεται — δηλαδή μόνο το εκάστοτε thread έχει πρόσβαση στις μεταβλητές υπολογισμού του.
5. Επανάληψη των 1-4 έως να τελειώσουν οι τιμές εισόδου
6. Η main περιμένει να ολοκληρώσουν όλα τα threads που υπολογίζουν ακόμη, και όταν γίνει αυτό σηματοδοτεί μέσω μιας «προσωπικής» μεταβλητής στο κάθε thread ότι πρέπει να τερματίσει.
7. Η main εξέρχεται της ρουτίνας της.

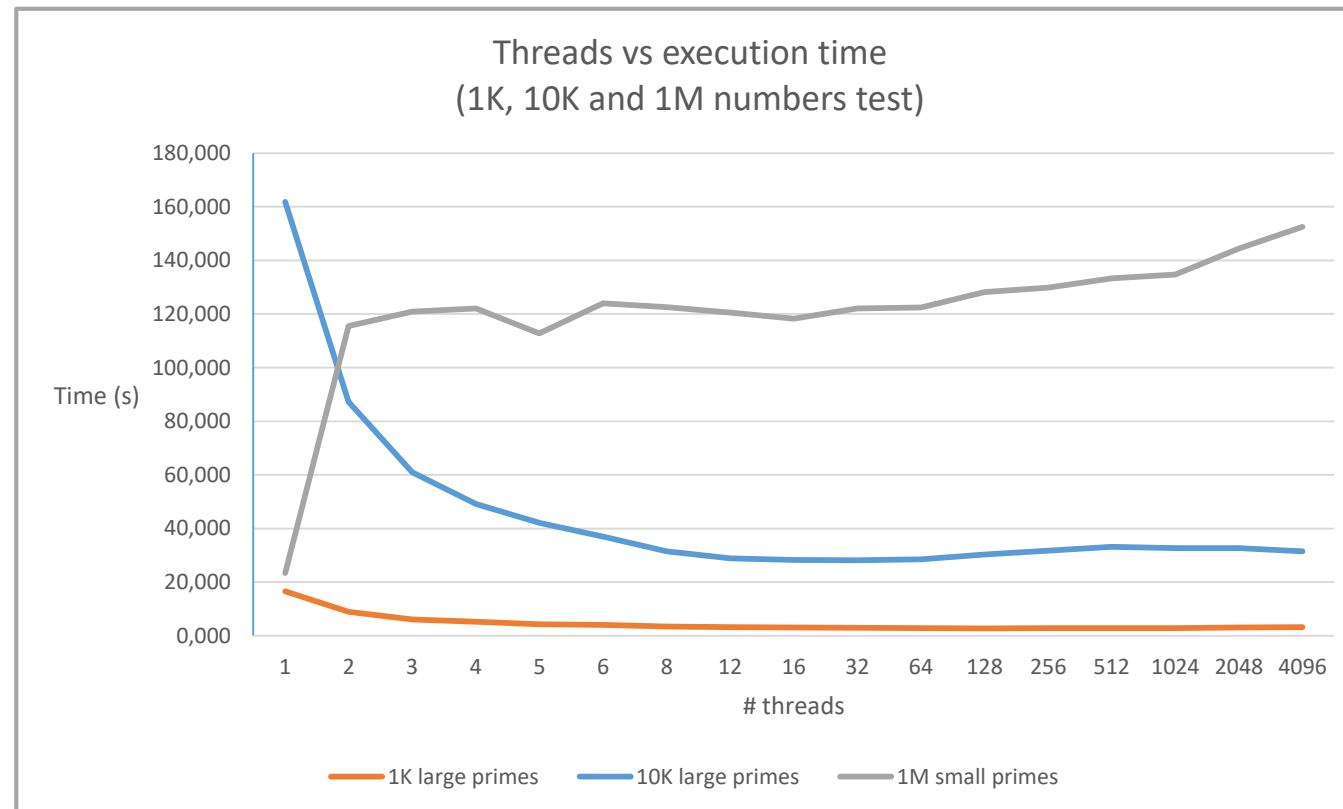
Επικοινωνία main-workers

Υλοποιούμε την επικοινωνία της main με τα worker threads έχοντας ένα struct για το κάθε thread (worker_props : short for “worker properties”), και περνώντας πληροφορίες στο struct τις οποίες διαβάζει τόσο το αντίστοιχο thread όσο και η main. Δηλαδή, κοινές μεταβλητές που είναι “encapsulated” σε ένα struct για επικοινωνία μεταξύ threads.



Performance metrics

Εκτελέσαμε το πρόγραμμα με διαφορετικές τιμές threads και διαφορετικές τιμές εισόδου. Με χρονόμετρο καταλήξαμε σε συμπεράσματα για την επιρροή του # threads και των τιμών εισόδου (αριθμός, πολυπλοκότητα υπολογισμού) στο χρόνο εκτέλεσης.



Performance metrics

Παρατηρούμε ότι ο χρόνος υπολογισμού μειώνεται ταχύτατα καθώς αυξάνουμε τα processing threads, όταν πρόκειται για είσοδο με «δύσκολους» αριθμούς που θέλουν πολλούς κύκλους για να βρεθεί αν είναι primes.

Παρόλα αυτά, για είσοδο περισσότερων αριθμών (1000000) αλλά πολύ πιο εύκολων στον υπολογισμό, παρατηρούμε αύξηση του execution time σε οποιαδήποτε λειτουργία πέρα από single-threaded. Αυτό πιθανότατα οφείλεται στο overhead που υπάρχει για τη δημιουργία και διαχείριση των threads από τη βιβλιοθήκη pthreads καθώς και το συνεχές context switching του CPU που συμβαίνει μεταξύ των workers.

Με άλλα λόγια, φαίνεται πως όταν έχουμε μεγάλο workload το οποίο όμως αποτελείται από lightweight επιμέρους εργασίες, δεν αξίζει το multithreading διότι το overhead της «γραφειοκρατίας» του threading system είναι περισσότερο από την δυσκολία υπολογισμού του ίδιου του workload.

Distributed primes calculation

Evangelos Zampras | Evangelos Balamotis

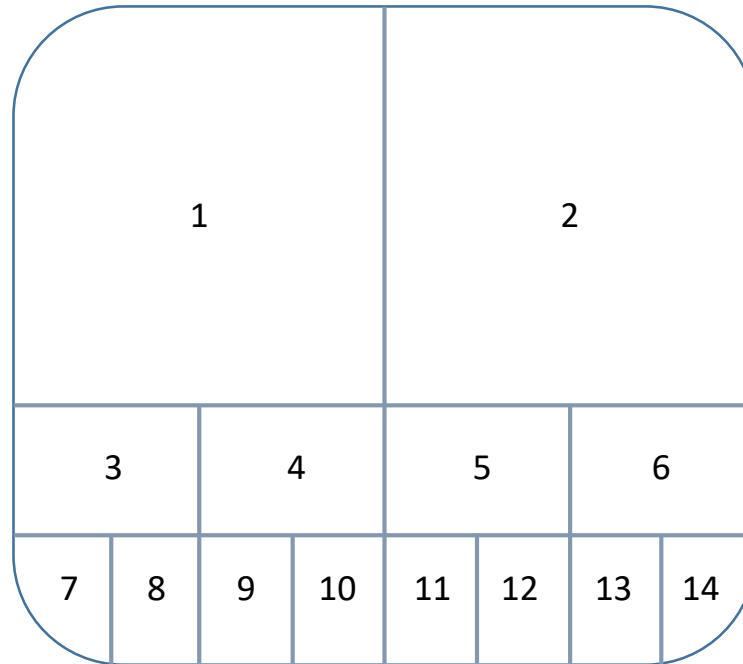
ECE, University of Thessaly

Thanks for your time!

Threaded External Merge Sort

Evangelos Zampras | Evangelos Balamotis

ECE, University of Thessaly



Υλοποίηση threaded merge sort

Η main δίνει το αρχείο εισόδου στο πρώτο worker thread. Στη συνέχεια, με βάση το specification, το thread εκτελεί internal merge sort αν το μέγεθος είναι μικρότερο από 64 ακέραιους (256 bytes σε μια μηχανή όπου `sizeof(int) == 4`), ή χωρίζει το αρχείο σε δύο μισά και αναθέτει το sorting του καθενός σε ένα sub-thread.

Όταν επιστρέφουν τα 2 threads, ενώνει τα αρχεία σε ένα sorted αρχείο εξόδου.

Race conditions

Με βάση τον τρόπο που έχει γραφτεί το πρόγραμμα, δεν υπάρχουν κίνδυνοι συγχρονισμού καθώς το κάθε thread ταξινομεί ένα ξεχωριστό κομμάτι του αρχείου και στη συνέχεια ένα μοναδικό άλλο thread (το parent thread και των δύο) ενώνει τα δύο υπο-αρχεία σε ένα.

Με αυτόν τον τρόπο αποφεύγουμε λιμοκτονία, απώλειας προόδου και deadlocks καθώς το πρόγραμμα χωρίζει εξ αρχής το input σε μέρη και το κάθε thread αναλαμβάνει μόνο ένα από αυτά κάθε χρονική στιγμή.

Αναδρομή και μνήμη

Με δοκιμές παρατηρήσαμε ότι όταν επιτρέπουμε sorting στη μνήμη μόνο σε μεγέθη μικρότερα από 64 ακεραίους (256 bytes), έχουμε περιορισμό στο πόσο μεγάλα αρχεία μπορούμε να ταξινομήσουμε αφού δημιουργείται stack overflow λόγω των πολλών επιπέδων αναδρομής.

Αν είμαστε πιο ανεκτικοί στο ελάχιστο μέγεθος μνήμης και το θέσουμε πάνω από μία γενναιόδωρη τιμή όπως 1 KB, μπορούμε να ταξινομήσουμε πολύ μεγαλύτερα αρχεία.

Threaded External Merge Sort

Evangelos Zampras | Evangelos Balamotis

ECE, University of Thessaly

Thanks for your time!